

Osciloscopio Básico Monocanal

Nestasio Facundo Nicolas
Técnicas Digitales III
UTN FRA
Avellaneda, Argentina
niconesta66@gmail.com

Canteros Sebastian
Técnicas Digitales III
UTN FRA
Avellaneda, Argentina
sebastiancanterosr@gmail.com

Abstract—El proyecto que detalla este informe consiste en el diseño e implementación de un osciloscopio de memoria digital monocanal. El instrumento se basa en un microcontrolador ESP32 ejecutando el sistema operativo en tiempo real FreeRTOS. Se propone una arquitectura de adquisición continua mediante Acceso Directo a Memoria (DMA) y un sistema de sincronismo implementado por software para permitir la visualización de la señal con función de predisparo. La interfaz de usuario consta de un encoder rotativo y transmisión de datos para visualización externa.

Palabras Clave—Osciloscopio, FreeRTOS, ESP32, DSO, DMA, Trigger.

ÍNDICE

I	FUNDAMENTACIÓN	1
I-A	Problemática	1
I-B	Solución	1
I-C	Fundamentación del Software	1
I-D	Fundamentación del Hardware	2
II	OBJETIVOS	2
III	ALCANCE	2
IV	DIAGRAMA EN BLOQUES	2
V	DIAGRAMA EN TAREAS	2
VI	HARDWARE	2
VI-A	ESP32-S3	2
VI-B	Etapas de atenuación	2
VI-C	Display Oled Ssd1306	3
VI-D	Encoder KY-040	3
VI-E	Esquemáticos	3
VII	SOFTWARE	4
VII-A	Lista de tareas	4
VII-B	Interfaz Gráfica	4
VIII	CONCLUSIÓN	4
	Bibliografía	4
	Anexo	6
	Anexo A: Diagramas del Sistema	6

Anexo B: Manual de Usuario y Dispositivo

10

I. FUNDAMENTACIÓN

A. Problemática

En el desarrollo y diagnóstico de sistemas electrónicos, es indispensable contar con instrumentos capaces de registrar y visualizar señales variables en el tiempo. Los osciloscopios de laboratorio tradicionales ofrecen altas prestaciones, pero su costo, tamaño y complejidad limitan su accesibilidad para entornos de desarrollo ágil o mediciones portátiles.

B. Solución

Frente a esta problemática, se propone el diseño de un osciloscopio digital compacto y de bajo costo utilizando el microcontrolador ESP32. Al trasladar funciones típicamente resueltas por hardware, como el sincronismo, al dominio del procesamiento por software, se reduce drásticamente la cantidad de componentes externos. Esto permite mantener una alta flexibilidad en la configuración y optimizar el presupuesto del instrumento, aprovechando el alto poder de cómputo y la accesibilidad del microcontrolador seleccionado.

C. Fundamentación del Software

Para el desarrollo de este proyecto se optó por utilizar el sistema operativo en tiempo real FreeRTOS. Este entorno facilita la organización de los flujos de trabajo y las rutinas en tareas independientes con prioridades asignadas, permitiendo una ejecución concurrente eficiente mediante el uso de mecanismos tales como colas de mensajes y semáforos de eventos.

En lo que respecta a la etapa de digitalización, el muestreo de la señal analógica se realiza mediante el conversor analógico-digital (ADC) nativo del microcontrolador, el cual posee una resolución de 12 bits. Considerando que los conversores de los osciloscopios digitales comerciales operan típicamente con una resolución de 8 bits a 16 bits, se determina que la resolución del dispositivo es satisfactoria para la tarea.

Por otro lado, la interfaz y las acciones del usuario se gestionan mediante interrupciones externas por GPIO, las cuales envían los eventos de configuración hacia el sistema a través de la cola `q_scale`. Finalmente, para la representación gráfica de la señal en el dominio del tiempo, los datos procesados son despachados por el puerto UART mediante una cola de

visualización, permitiendo su ploteo en tiempo real a través de un software hecho por Python utilizando la librería pyqtgraph.

D. Fundamentación del Hardware

Dado que los pines del conversor analógico-digital del microcontrolador operan estrictamente en el rango de 0 V a 3.3 V y no admiten tensiones negativas, es necesario implementar una etapa previa de adaptación. Esta etapa (compuesta por atenuadores, amplificadores operacionales y divisores resistivos) cumple dos funciones críticas: escalar la amplitud de la señal de entrada para que coincida con la escala vertical seleccionada por el usuario, y sumar un nivel de tensión continua (offset) de 1.65 V. De este modo, una señal alterna pura oscilará de manera segura dentro de la ventana de lectura del microcontrolador, protegiendo el silicio contra sobretensiones y permitiendo la reconstrucción matemática posterior del semiciclo negativo.

El núcleo de la digitalización explota las capacidades del ADC interno de 12 bits de resolución operando de forma ininterrumpida. Al configurar el ADC para que sea disparado constantemente por un temporizador por hardware, el controlador de Acceso Directo a Memoria (DMA) transfiere las muestras hacia arreglos de memoria RAM de manera automática y cíclica. Al operar con este flujo continuo de datos, el sistema delega la decisión de captura a la tarea de sincronismo (Trigger); es esta lógica la que evalúa la memoria en tiempo real y, al detectar el evento de interés, emite la orden de congelar y guardar el búfer actual para su procesamiento.

Para la interfaz de control se optó por utilizar un encoder que envíe las entradas del usuario a través de GPIO y una pantalla OLED donde el usuario pueda visualizar la configuración, el impacto de su entrada y los datos de interés. La señal se visualizará a través de una PC que recibirá los datos de la señal por UART.

II. OBJETIVOS

- Procesar y transmitir señales analógicas para su visualización a través de un osciloscopio digital de forma segura.
- Implementar un trigger por nivel configurable con selección de flanco.
- Implementar dos escalas verticales seleccionables.
- Implementar tres bases de tiempo seleccionables.
- Visualizar la señal en una PC a través de un software de visualización.

III. ALCANCE

Se pretende lograr un osciloscopio digital monocanal con los siguientes parámetros:

- Escalas Verticales (Amplitud): 1 V/div y 5 V/div.
- Escalas Horizontales (Base de tiempo): 1 ms/div, 2 ms/div y 5 ms/div.
- Atenuación de entrada: x1 y x10

IV. DIAGRAMA EN BLOQUES

En la Figura 10 del Anexo podemos ver el esquema en bloques del osciloscopio, donde se detallan las entradas y salidas de los bloques y sus conexiones. Aquellos en los que no se especifican las entradas y salidas es porque se detallan en el diagrama de tareas.

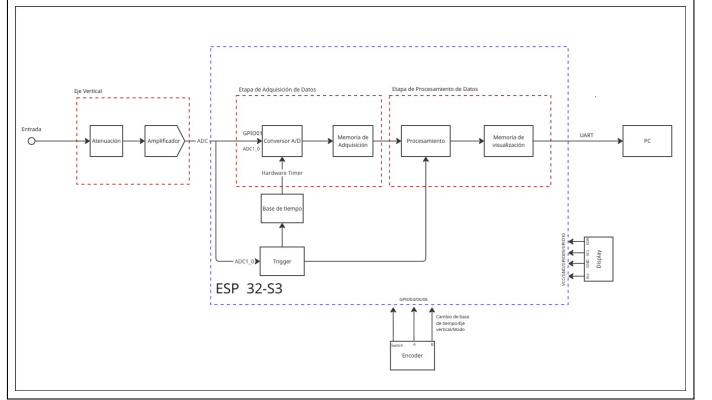


Fig. 1. Diagrama en bloques.

V. DIAGRAMA EN TAREAS

En la Figura 13 del Anexo podemos ver el diagrama de tareas de FreeRTOS, donde se detalla la funcionalidad de cada una de ellas, sus prioridades y su interacción con las demás mediante colas y eventos registrados en semáforos.

VI. HARDWARE

A. ESP32-S3

- Microcontrolador de *Espressif Systems* con arquitectura Xtensa LX7 de doble núcleo.
- Frecuencia de funcionamiento máxima de 240 MHz.
- Tensión de alimentación de 3.0 V a 3.6 V.
- Hasta 45 GPIO configurables.
- Incorpora convertidores analógico-digitales (ADC) de 12 bits, utilizados para la adquisición de las señales de entrada del osciloscopio.
- Interfaces de comunicación UART, SPI, I2C, I2S, CAN (TWAI) y USB OTG.

B. Etapa de atenuación

Para la etapa de atenuación X1 se podrán medir tensiones de hasta 3.3V sin divisor resistivo. Para llegar a 33V se utilizará un divisor resistivo debido a que el ADC del ESP32-S3 es de hasta 3.3V.

$$33V \frac{R2}{R1 + R2} = 3.3V \quad (1)$$

$$R2 = 10Kohm \quad (2)$$

$$R1 = 90Kohm \quad (3)$$

Además, para poder soportar las señales negativas se debe implementar una señal del offset, para eso se utilizará el pin de 3.3V del micro y un operacional en la configuración de

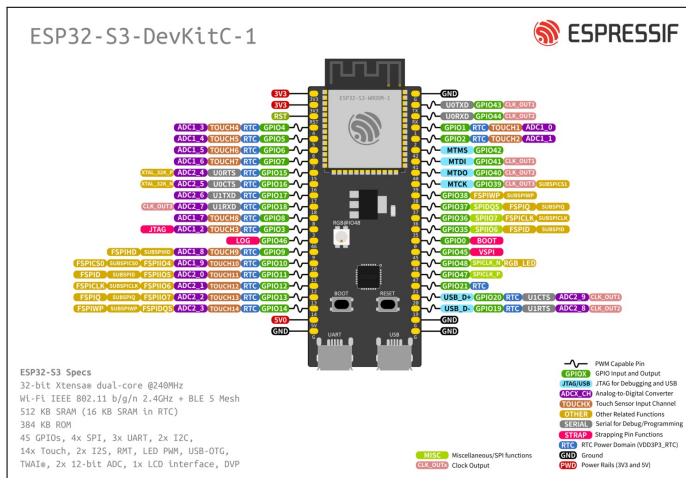


Fig. 2. ESP32-S3 Pinout.

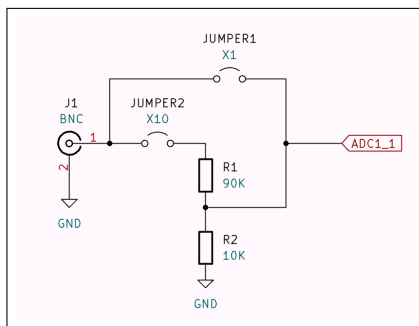


Fig. 3. Etapa de atenuación.

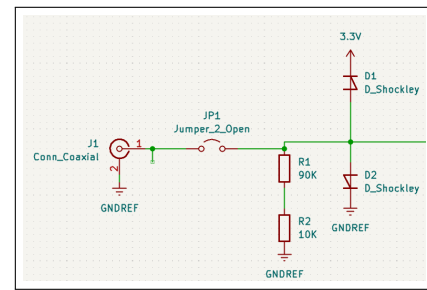


Fig. 5. Etapa de proteccion.

- Resolución de 128x64 pixeles.
- Pantalla de 35x24 mm.

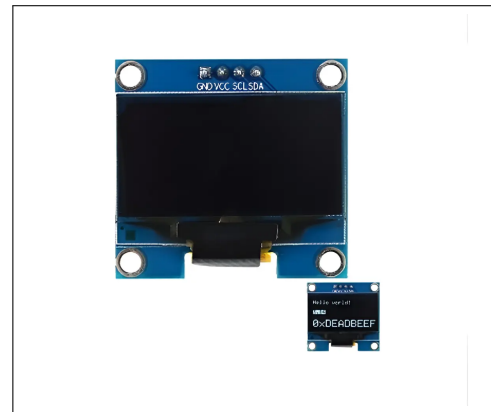


Fig. 6. Display Oled Ssd1306.

sumador no inversor.El operacional utilizado sera el TL074 Y se utilizara el siguiente circuito:

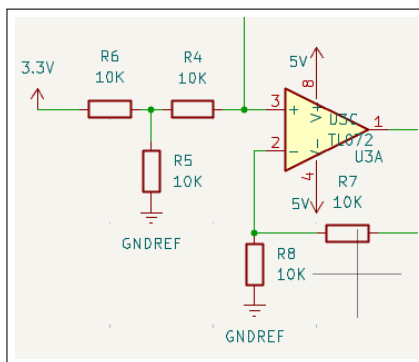


Fig. 4. Enter Caption

Para el circuito de protección contra sobretensiones, utilizaremos dos diodos shockley uno conectado a una tensión de referencia de 3.3V ,y otro conectado a masa, esto hará que cuando la señal sea negativa, limitara la tensión de entrada a 0,2 V y cuando la señal extienda el rango del osciloscopio la limitara a 3.5V.

C. Display Oled Ssd1306

- Tensión de alimentación de 3.3 V.

D. Encoder KY-040

Para la seleccion de escalas,de modos y de niveles se utilizo el encoder KY-40

- Voltaje de Operación: 5V
- 20 pulsos por revolución..
- Encoder incremental con pulsador.

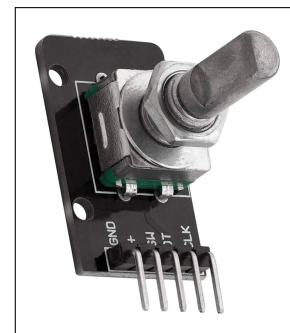


Fig. 7. KYO-040.

E. Esquemáticos

Se realizo el circuito en KiCad con los jumpers para cambio de escala,las etapas de atenuación y circuitos de protección.

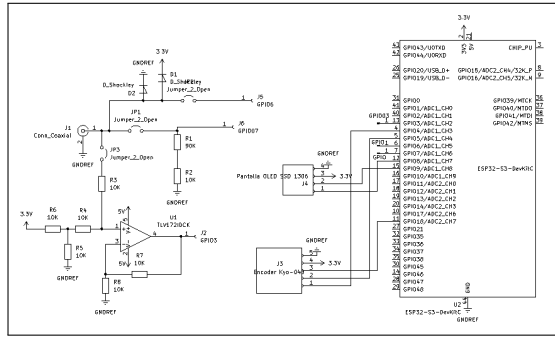


Fig. 8. Esquemático del circuito

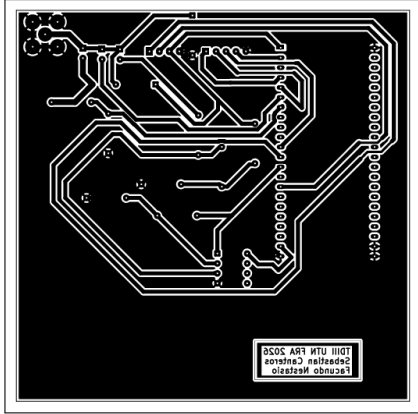


Fig. 9. PCB

VII. SOFTWARE

A. Lista de tareas

El flujo de datos se diseñó para separar las etapas de adquisición, procesamiento y comunicación:

- **Gestión de datos:** La tarea `vAdcTask` envía punteros de memoria a `vTriggerTask` a través de la cola `full_queue` para maximizar la velocidad de transferencia.
- **Parámetros de configuración:** Se implementó un patrón de diseño tipo *Mailbox* mediante la cola `config_queue`, permitiendo que la tarea del menú actualice los parámetros (flanco, nivel de disparo, escalas) de forma segura y no bloqueante para el motor de procesamiento.
- **Comunicación asíncrona:** La tarea `vUartTask` actúa como un portero de la comunicación, recibiendo tramas completas desde `vTriggerTask` a través de `uart_queue`, evitando colisiones de datos y garantizando la fluidez de la transmisión hacia el equipo externo.

Para cumplir con los requerimientos de captura de señales de alta velocidad, se ha configurado una frecuencia de muestreo de 20 kHz. Esta tasa asegura una fidelidad óptima para la representación de las señales, permitiendo realizar el análisis de disparo y filtrado de ruido de manera eficiente antes de la visualización.

TABLE I
DESCRIPCIÓN DE TAREAS

Tarea	Descripción
<code>vAdcTask</code>	Adquiere muestras crudas del conversor mediante DMA y gestión de buffers ping-pong.
<code>vTriggerTask</code>	Detecta el flanco de disparo y encuadra la ventana de datos para su transmisión.
<code>vUartTask</code>	Gestiona la transmisión asíncrona de tramas empaquetadas hacia la interfaz externa.
<code>vMenuTask</code>	Gestiona la interfaz de usuario, navegación del menú y control de parámetros del sistema.
<code>vButtonTask</code>	Implementa el filtrado de rebotes (debounce) y procesa eventos de entrada del usuario.

B. Interfaz Gráfica

Para el ploteo de las señales se optó finalmente por utilizar una interfaz desarrollada en Python y la librería `pyqtgraph` dado que las otras opciones como el `SerialPlot` restringían mucho la visualización. Además poseemos más libertad con Python para desarrollar una interfaz a nuestro gusto con cursores y valores pico.

La interfaz se caracteriza por:

- **Sincronización Dinámica:** Implementa un protocolo basado en una cabecera `SYNC`, permitiendo actualizar parámetros de control (flanco, escalas) de forma instantánea según el estado del firmware.
- **Visualización Eficiente:** Utiliza un gestor de eventos (`QTimer`) para el refresco del gráfico, asegurando una visualización fluida incluso bajo flujos constantes de datos.
- **Análisis de Señales:** Incorpora cursores dinámicos para la medición precisa de parámetros clave como tensión pico-pico, máximos y mínimos sobre la señal en tiempo real.
- **Gestión de Errores:** Incluye un sistema de detección de pérdida de enlace que notifica al usuario ante cualquier interrupción en la comunicación serial.

VIII. CONCLUSIÓN

Se lograron abordar todos los ítems planteados. Se identificaron los límites del ESP32 en cuanto al uso del ADC, lo que requirió adaptar la entrada mediante un atenuador resistivo y, en otros casos, emplear diferentes métodos de acondicionamiento de señal. Además, se aprovecharon las ventajas de contar con un sistema operativo en tiempo real como FreeRTOS, que permite sincronizar las tareas para garantizar el correcto funcionamiento del dispositivo, a diferencia de las funciones convencionales.

Asimismo, se utilizaron diversas características del ESP32-S3: el PCNT para la lectura del encoder, el ADC continuo de 12 bits para la adquisición de la señal de entrada y el UART para establecer una comunicación confiable con la aplicación.

BIBLIOGRAFÍA

- [1] H. Yavuz, *SerialPlot - Realtime Plotting Software*, 2023. [En línea]. Disponible en: <https://hackaday.io/project/5334-serialplot-realtime-plotting-software>

- [2] Espressif Systems, “ESP32-S3 Wi-Fi & BLE 5 SoC,” [En línea].
Disponible en: <https://www.espressif.com/en/products/socs/esp32-s3>

ANEXO

ANEXO A: DIAGRAMAS DEL SISTEMA

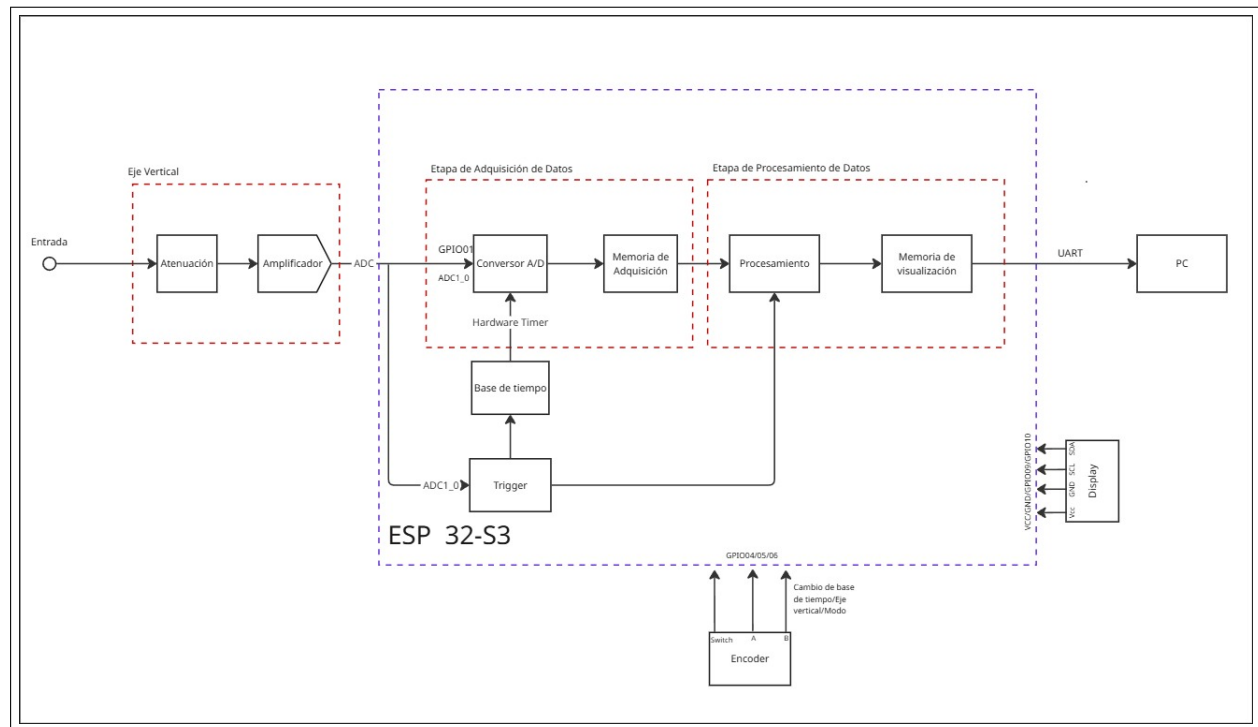


Fig. 10. Diagrama en bloques.

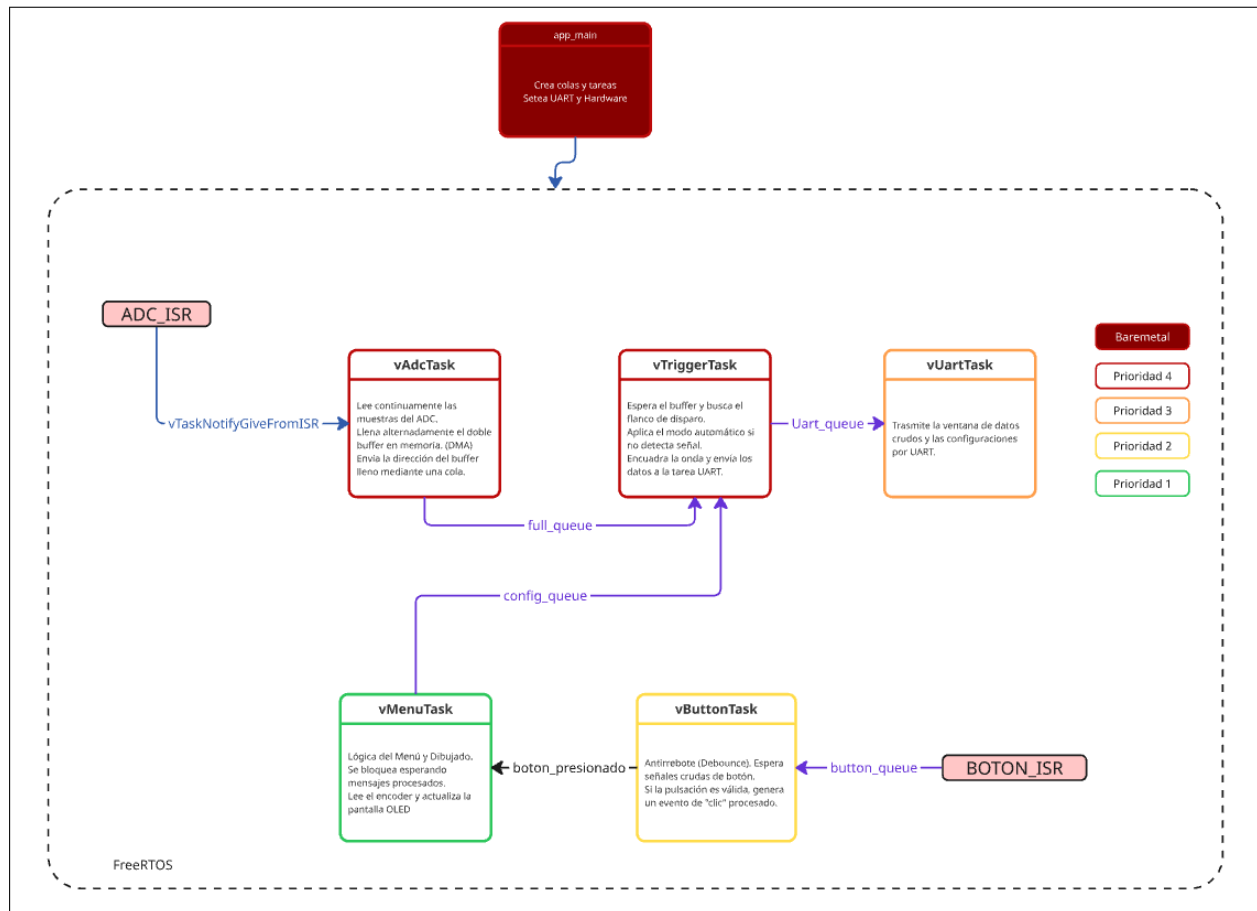


Fig. 11. Diagrama de tareas del osciloscopio.

ANEXO B: ESQUEMÁTICO Y PCB

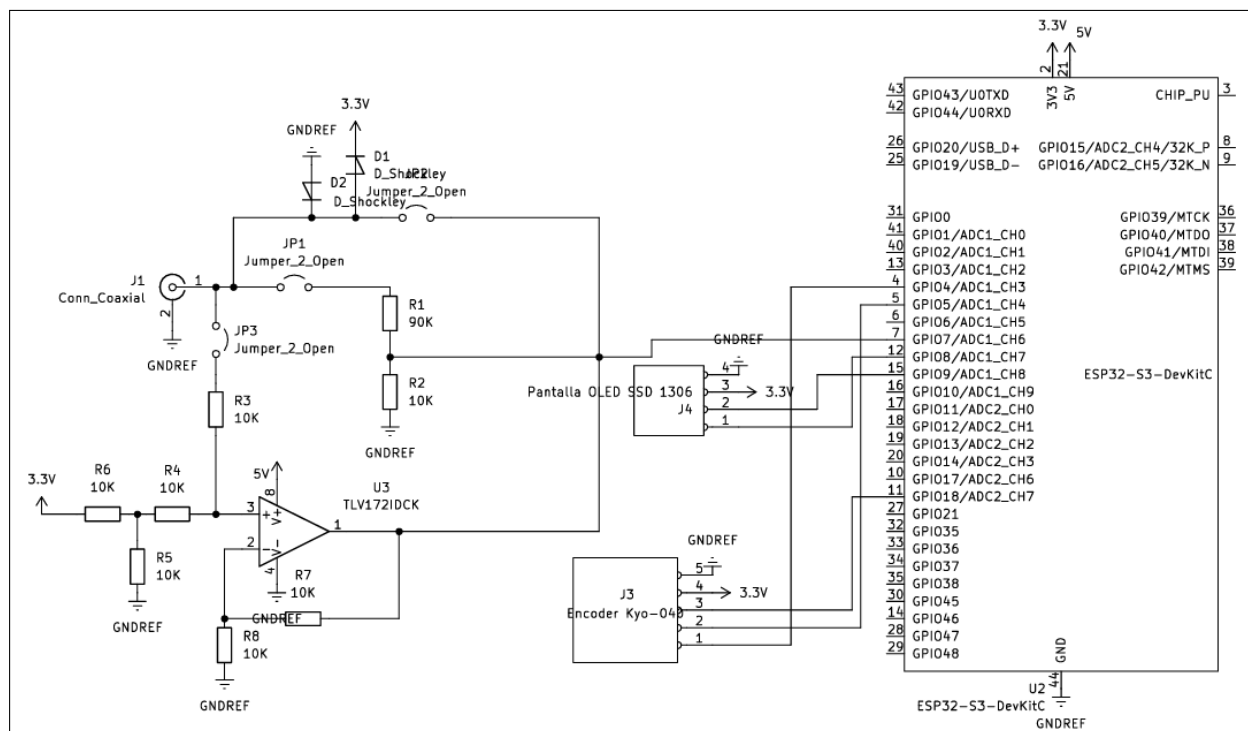


Fig. 12. Esquemático.

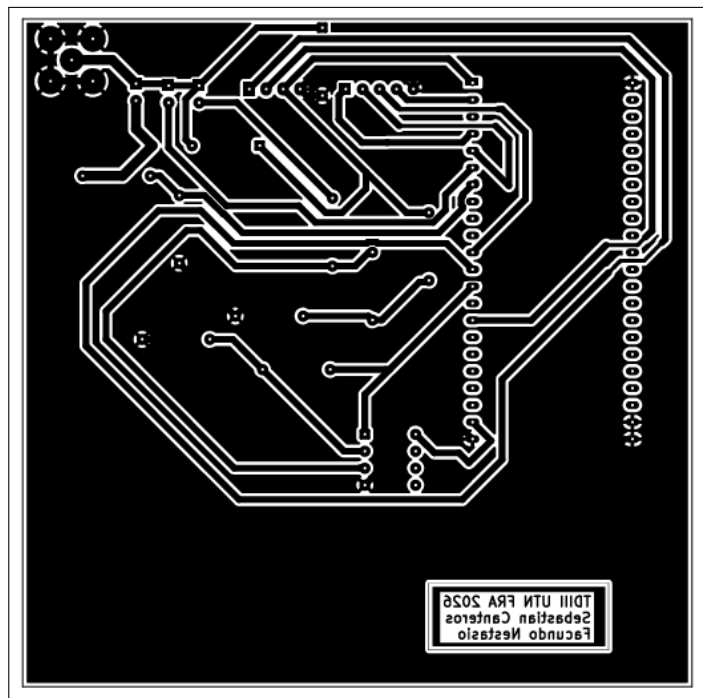


Fig. 13. PCB

ANEXO C: MANUAL DE USUARIO Y DISPOSITIVO

Descripción de uso: En este apartado se explicará el funcionamiento del dispositivo desarrollado. El osciloscopio cuenta con solo un elemento de control, que será el encoder. Este permitirá navegar por el menú y seleccionar las opciones deseadas. Además, la pantalla del instrumento funcionará como interfaz máquina-usuario para un control más ameno. A continuación se detallarán cada una de las opciones del menú:



Fig. 14. Placa final.

- **Amplitud/div:** Con esta opción se podrá cambiar la escala del eje vertical, teniendo 2 opciones posibles: 1V/div y 5V/div.
- **tiempo/div:** Se podrá alternar la escala horizontal con hasta 3 escalas diferentes: 1ms/div, 2ms/div y 5ms/div.
- **Nivel de trigger:** En este apartado se puede variar el nivel de trigger para mantener el sincronismo con la señal.
- **Flanco:** Se puede elegir en qué momento disparará el trigger, ya sea en la subida o en la bajada.
- **Software:** El programa utilizado solo sirve para visualizar la medición realizada, sin ningún tipo de control por PC.

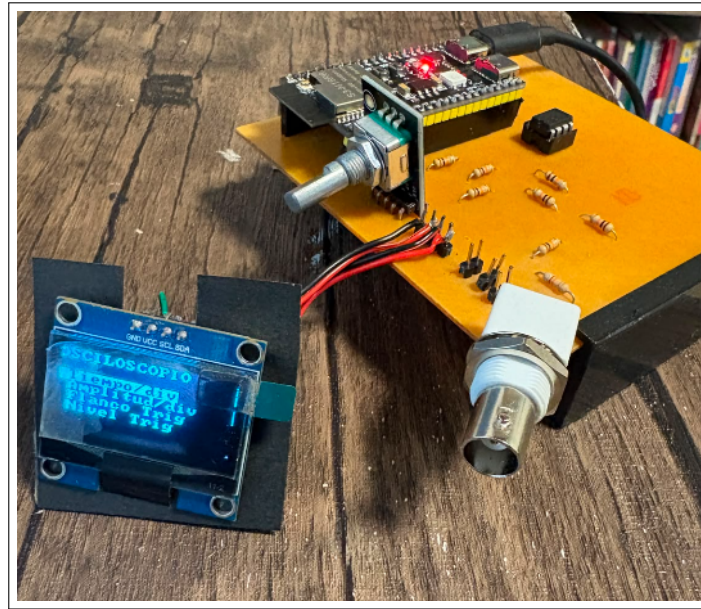


Fig. 15. Placa final.

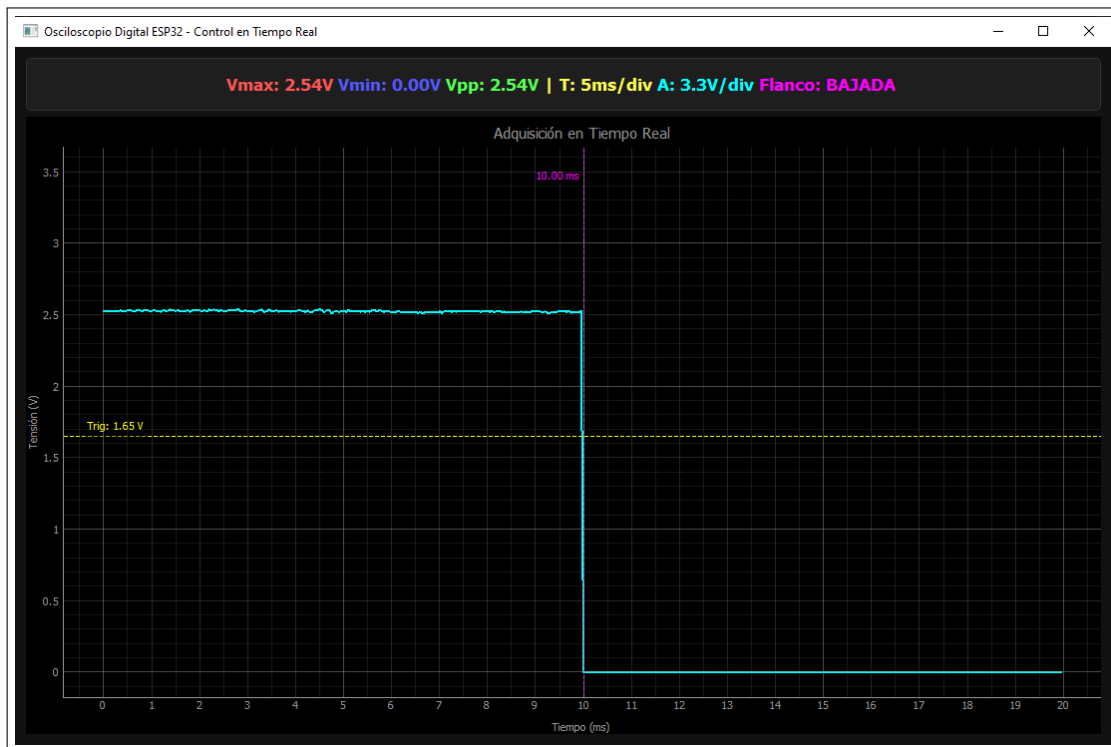


Fig. 16. Interfaz gráfica del osciloscopio hecha con Python.